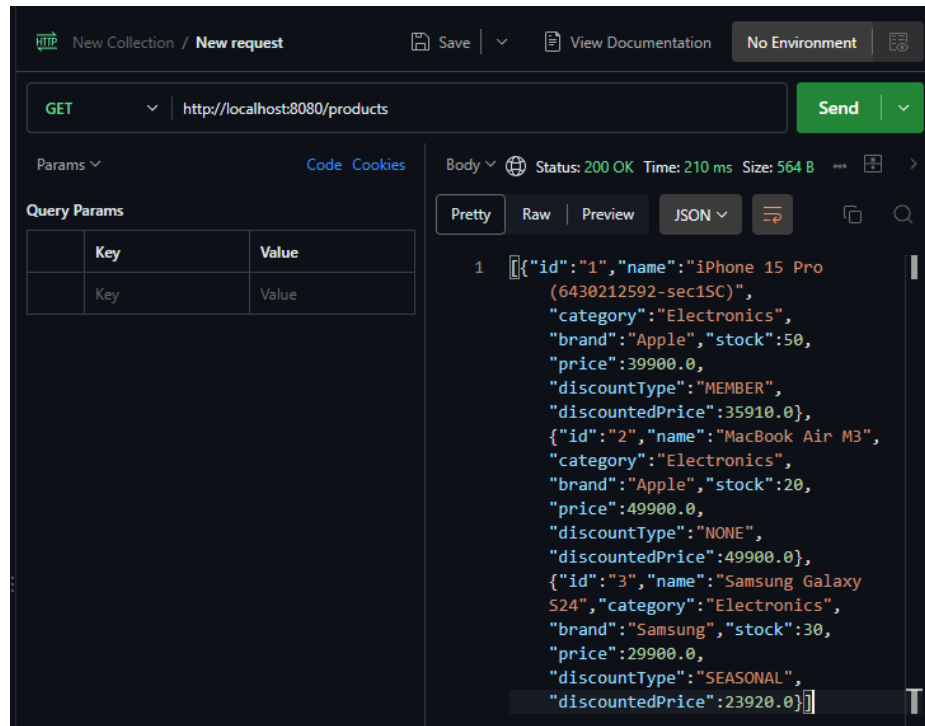


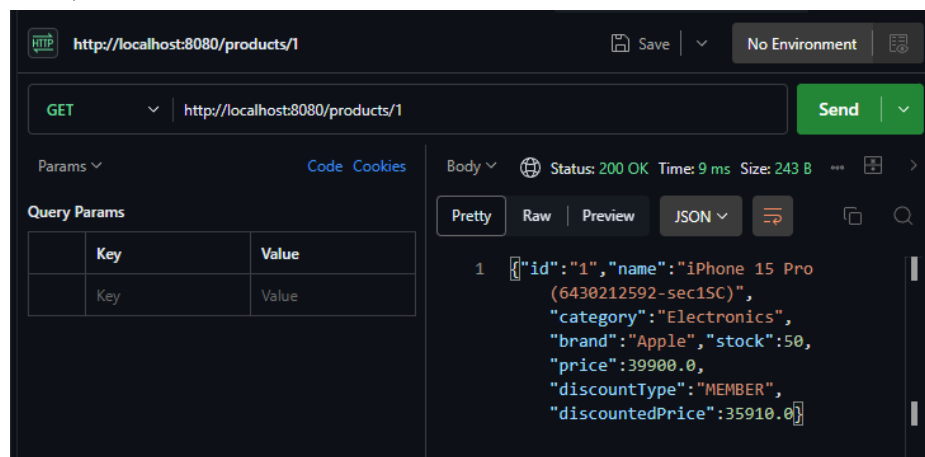
Lab10_6430212592 Sec1SC

นางสาวอาทิตย์ยา โคตรธิสาร 643021259-2 Section.1SC

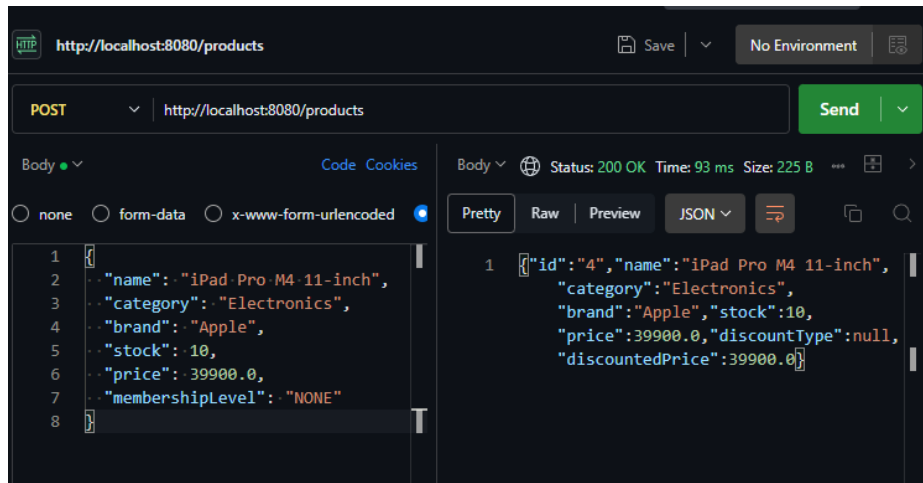
- Screenshot
 - GET /products



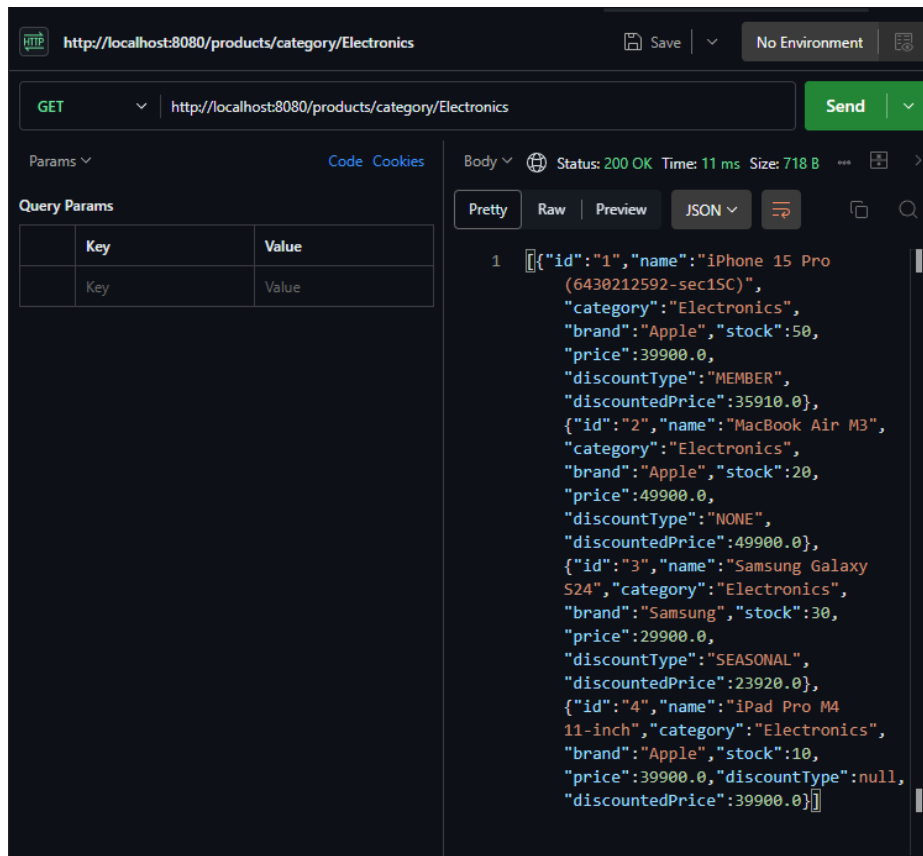
- GET /products/1



- POST /products



- GET /products/category/Electronics



- GET /products/1/price

HTTP `http://localhost:8080/products/1/price` Save No Environment

GET `http://localhost:8080/products/1/price` Send

Params Code Cookies

Query Params

	Key	Value
	Key	Value

Body Status: 200 OK Time: 11 ms Size: 77 B

Pretty Raw Preview JSON

```
1 35910.0
```

- DELETE /products/2

HTTP `http://localhost:8080/products/2` Save No Environment

DELETE `http://localhost:8080/products/2` Send

Params Code Cookies

Query Params

	Key	Value
	Key	Value

Body Status: 200 OK Time: 11 ms Size: 38 B

Pretty Raw Preview Text

```
1
```

- GET /products เช็คผลหลังลบ

HTTP `http://localhost:8080/products` Save No Environment

GET `http://localhost:8080/products` Send

Params Code Cookies

Query Params

	Key	Value
	Key	Value

Body Status: 200 OK Time: 9 ms Size: 567 B

Pretty Raw Preview JSON

```
1 [{"id": "1", "name": "iPhone 15 Pro (6430212592-sec15C)", "category": "Electronics", "brand": "Apple", "stock": 50, "price": 39900.0, "discountType": "MEMBER", "discountedPrice": 35910.0}, {"id": "3", "name": "Samsung Galaxy S24", "category": "Electronics", "brand": "Samsung", "stock": 30, "price": 29900.0, "discountType": "SEASONAL", "discountedPrice": 23920.0}, {"id": "4", "name": "iPad Pro M4 11-inch", "category": "Electronics", "brand": "Apple", "stock": 10, "price": 39900.0, "discountType": null, "discountedPrice": 39900.0}]
```

- Reactive Programming vs Blocking — ต่างกันอย่างไร ?
 - **Blocking (Synchronous):** เป็นการทำงานแบบดั้งเดิมที่เวลาเขียนโค้ด Thread จะต้องหยุดรอการทำงานของ I/O เสมอ เช่น ยิงไปดึงข้อมูลจาก Database หรือ API อื่น Thread นี้จะว่างงานและถูกบล็อกไว้จนกว่าจะได้ข้อมูลกลับมา ถ้ามีคนเข้าเว็บพร้อมกันเยอะๆ Thread Pool อาจจะเต็มและทำให้ระบบค้างหรือหน่วงได้
 - **Reactive Programming (Non-blocking):** เป็นการทำงานแบบยิงคำสั่งไปแล้วไม่ต้องยืนรอ Thread สามารถสลับไปให้บริการ Request ของคนอื่นต่อได้เลย พอ Database ประมวลผลเสร็จแล้วค่อยส่งสัญญาณ (Event/Callback) กลับมาบอกให้ระบบทำงานในขั้นต่อไป วิธีนี้ทำให้เราใช้ทรัพยากร Server คุ่มค่าขึ้น รองรับผู้ใช้งานได้เยอะมากโดยใช้ Thread แค่นิดเดียว
- Mono vs Flux — ใช้กรณีไหน และทำไม
 - **Mono (Mono<T>):** ใช้กับข้อมูลที่มี 0 หรือ 1 รายการ เท่านั้น เหมาะกับการทำอะไรที่ให้ผลลัพธ์เดียวชั่วคราว เช่น ค้นหาสินค้าด้วย ID (เจอ 1 ชิ้น หรือไม่เจอเลยก็จะคืนค่าว่าง), การบันทึกข้อมูล (เซฟเสร็จคืนค่า Object มา 1 ตัว) หรือใช้ Mono<Void> สำหรับการลบที่ไม่มีการคืนค่าอะไรเลย
 - **Flux (Flux<T>):** ใช้กับข้อมูลที่มี ตั้งแต่ 0 ถึง N รายการ (หลายรายการ) เหมาะกับการดึงข้อมูลออกมาเป็นกลุ่มหรือเป็น List เช่น การดึงรายการสินค้าทั้งหมดในระบบ หรือดึงข้อมูลแบบ Stream ที่ทยอยไหลเข้ามาเรื่อยๆ
- WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()
 - **.get():** (หรือ .post(), .delete()) เป็นการบอกว่าจะยิง Request ด้วย HTTP Method อะไร
 - **.uri():** เอาไว้ใส่ URL ปลายทางที่ต้องการยิงไปหา และสามารถแนบตัวแปรใส่เข้าไปใน Path ได้ด้วย เช่น .uri("/products/{id}", id)
 - **.retrieve():** เป็นตัวคำสั่งที่สั่งให้ WebClient เริ่มยิง Request ออกไปจริงๆ (ลั่นไก) และเตรียมพร้อมดึงเอาข้อมูล Response ที่ปลายทางตอบกลับมา
 - **.bodyToMono(T.class) / .bodyToFlux(T.class):** หลังจากได้ Response มาแล้ว คำสั่งนี้จะเอาส่วน Body (ที่เป็น JSON) มาแปลงร่างเป็น Java Object คลาสที่เรากำหนด (T) โดยห่อไว้ใน Mono (ถ้าข้อมูลมีชิ้นเดียว) หรือ Flux (ถ้ามาเป็น Array หลายชิ้น) เพื่อให้เราเอาตัวแปรไปเขียนต่อในระบบได้

- Code ของ Repository, Service, Controller พร้อม comment อธิบาย

ProductRepository.java

```
package com.cp.lab10.repository;

import com.cp.lab10.model.Product;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import org.springframework.stereotype.Repository;

@Repository
public class ProductRepository {

    // ใช้ ConcurrentHashMap จำลองเป็น Database เก็บข้อมูลไว้ใน Memory ชั่วคราว[cite: 3]
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    public ProductRepository() {
        // ใส่ข้อมูลเริ่มต้น (Seed data) โดยขึ้นแรกใส่เป็นรหัสนักศึกษาตัวเอง[cite: 3]
        store.put("1", new Product("1", "iPhone 15 Pro (6430212592-sec1SC)", "Electronics", "Apple", 50, 39900.0,
"MEMBER"));
        store.put("2", new Product("2", "MacBook Air M3", "Electronics", "Apple", 20, 49900.0, "NONE"));
        store.put("3", new Product("3", "Samsung Galaxy S24", "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));
    }

    // ค้นหาสินค้าด้วย ID: ห่อด้วย Mono.justOrEmpty() ป้องกันพังถ้าหาค่าไม่เจอแล้วได้ null กลับมา[cite: 3]
    public Mono<Product> findById(String id) {
        return Mono.justOrEmpty(store.get(id));
    }

    // ค้นหาสินค้าทั้งหมด: แปลงข้อมูลทั้งหมดใน Map ให้กลายเป็น Stream หลายๆ ตัวด้วย Flux.fromIterable[cite: 3]
    public Flux<Product> findAll() {
        return Flux.fromIterable(store.values());
    }
}
```

ProductRepository.java (ต่อ)

```
// บันทึกสินค้า: เอาของใหม่ใส่ลงใน Map แล้วส่งข้อมูลที่เซฟเสร็จกลับไปเป็น Mono ก้อนเดียว[cite: 3]
public Mono<Product> save(Product product) {
    store.put(product.getId(), product);
    return Mono.just(product);
}

// ลบสินค้า: ลบตาม ID ที่ได้มา การลบไม่ได้คืนค่าอะไรกลับไป เลยใช้ Mono.empty() (Void)[cite: 3]
public Mono<Void> deleteById(String id) {
    store.remove(id);
    return Mono.empty();
}

// ค้นหาตามหมวดหมู่: ดึงสินค้าทั้งหมดมาก่อน แล้วเอามา .filter() เช็คว่า Category ตรงกับที่รับมาไหม[cite: 3]
public Flux<Product> findByCategory(String category) {
    return Flux.fromIterable(store.values())
        .filter(product -> product.getCategory().equalsIgnoreCase(category));
}
}
```

ProductService.java

```
package com.cp.lab10.service;

import com.cp.lab10.model.Product;
import com.cp.lab10.repository.ProductRepository;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

@Service
public class ProductService {

    // รับ Repository เข้ามาผ่าน Constructor Injection (ให้ Spring จัดการให้)[cite: 4]
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    // ดึง 1 รายการ: ถ้าหาใน Repository ไม่เจอ (Empty) จะสลับไปโยน Error ทิ้งไปเลยด้วย .switchIfEmpty()[cite: 4]
    public Mono<Product> getById(String id) {
        return repository.findById(id)
            .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));
    }

    // ดึงทั้งหมด: ไม่ต้องทำอะไรซับซ้อน โยนต่อให้ Repository ดึงข้อมูลเป็น Flux มาเลย[cite: 4]
    public Flux<Product> getAll() {
        return repository.findAll();
    }

    // บันทึกข้อมูล: เช็คว่ามี ID ส่งมาไหม ถ้าว่างแปลว่าเป็นของใหม่
    // เราจะใช้วิธีนับจำนวนสินค้าทั้งหมด (count) แล้วเอามาบวก 1 เป็น ID อันต่อไป (Auto-increment แบบง่ายๆ ทำแบบ)
    public Mono<Product> save(Product product) {
        if (product.getId() == null || product.getId().isEmpty()) {
            return repository.findAll().count().flatMap(count -> {
                product.setId(String.valueOf(count + 1));
                return repository.save(product);
            });
        }
        return repository.save(product);
    }
}
```

ProductService.java (ต่อ)

```
// ลบข้อมูล: ส่ง ID ไปให้ Repository ลบ[cite: 4]
public Mono<Void> delete(String id) {
    return repository.deleteByid(id);
}

// กรองตามหมวดหมู่: โยนต่อให้ Repository กรองหมวดหมู่ตาม String ที่รับมา[cite: 4]
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// คำนวณราคาหลังหักส่วนลด: หา Product ตัวนั้นออกมาก่อน แล้วใช้ .map() สกัดเอาแต่ตัวเลขราคา discountedPrice คืนกลับไป[cite: 4]
public Mono<Double> getDiscountedPrice(String id) {
    return getByid(id).map(p -> p.getDiscountedPrice());
}
}
```

ProductController.java

```
package com.cp.lab10.controller;

import com.cp.lab10.model.Product;
import com.cp.lab10.service.ProductService;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    // GET /products/{id} -> รับ PathVariable 'id' จาก URL แล้วไปเรียก Service ดึงของ 1 ชิ้น[cite: 5]
    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id);
    }

    // GET /products -> เรียก Service เพื่อดึง List สินค้าทั้งหมดออกมาเป็น Flux[cite: 5]
    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }

    // POST /products -> รับ JSON จาก Request Body แปลงเป็น Product แล้วส่งให้ Service เซฟลงคลัง[cite: 5]
    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {
        return service.save(product);
    }
}
```

ProductController.java (ต่อ)

```
// DELETE /products/{id} -> รับ ID เข้ามา แล้วส่งให้ Service ลบสินค้านั้นทิ้งไป[cite: 5]
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}

// GET /products/category/{category} -> รับชื่อหมวดหมู่จาก URL แล้วส่งให้ Service ช่วยกรองให้[cite: 5]
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}

// GET /products/{id}/price -> รับ ID เข้าไป แล้วส่งกลับมาเป็น Mono<Double> ที่มีแค่ตัวเลขราคา[cite: 5]
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id);
}
}
```